

Corrections complètes

Examens Internet et Bases de Données — Master Langue & Informatique 1ère année

Examen 2023	Serveur de messages TCP	15 mai 2023
Examen 2024	Copie de répertoire UDP	28 mai 2024
Examen 2025	Bibliothèque TCP + Serveur heure UDP	19 mai 2025

■ Structure commune : Partie 1 = Réseaux (5 pts) | Partie 2 = Programmation Java (15 pts). Les deux parties sont indépendantes.

■ Obligation de soumettre une archive .zip contenant UNIQUEMENT les fichiers .java (pas les .class). Notez le lien et l'heure d'envoi sur votre copie.

Méthode générale pour réussir l'examen

Ordre de traitement recommandé

Étape 1 — Partie réseaux (dès le début, ~15 min)

Lancez immédiatement les commandes réseau (nslookup, tracert/traceroute) dans le terminal pendant que vous créez votre projet Java. Les résultats mettent parfois du temps à arriver.

Étape 2 — Création du projet Java

Nommez-le IBD20XX_NomDeFamille. Créez une classe Client.java, une classe Serveur.java et une classe de test par jeu de test.

Étape 3 — Progression logique des questions

Les questions suivent le protocole pas à pas : traitez-les dans l'ordre. Chaque paire client/serveur est testée avant de passer à la suivante.

Étape 4 — Envoi de l'archive

Archive .zip avec UNIQUEMENT les .java. Notez le lien et l'heure sur la copie.

Commandes réseau (Partie 1)

```
# Adresse IP d'une URL
nslookup www.exemple.gov.xx      (Windows/Linux/Mac)
host www.exemple.gov.xx          (Linux/Mac)
```

```
# Classe IPv4 (1er octet de l'adresse)
0-127    → Classe A
128-191  → Classe B
192-223  → Classe C
224-239  → Classe D (multicast)
```

```
# Liste des routeurs
tracert www.exemple.gov.xx      (Windows)
traceroute www.exemple.gov.xx   (Linux/Mac)
```

TCP vs UDP en Java

TCP — connexion fiable, orientée flux :

```
// Serveur TCP
ServerSocket ss = new ServerSocket(port);
Socket client = ss.accept();
BufferedReader in = new BufferedReader(new InputStreamReader(client.getInputStream()));
PrintWriter out = new PrintWriter(client.getOutputStream(), true);
```

```
// Client TCP
Socket socket = new Socket("localhost", port);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

UDP — sans connexion, par datagrammes :

```
// Serveur UDP
DatagramSocket socket = new DatagramSocket(port);
byte[] buf = new byte[1024];
DatagramPacket p = new DatagramPacket(buf, buf.length);
socket.receive(p); // bloquant
```

```
// Client UDP
```

```
DatagramSocket socket = new DatagramSocket();  
InetAddress addr = InetAddress.getByName("localhost");  
byte[] buf = "message".getBytes();  
DatagramPacket p = new DatagramPacket(buf, buf.length, addr, port);  
socket.send(p);
```

Examen 2023 — Serveur de messages (TCP)

15 mai 2023 · Protocole TCP · 4 opérations : inscription, désinscription, envoi, lecture

Partie 1 — Réseaux [5 pts]

URL : <http://www.anrseb.gov.et>

Q1 — Champs de l'URL :

http → protocole de communication (HTTP)
www → sous-domaine (World Wide Web)
anrseb → nom d'hôte / domaine principal
gov → domaine de second niveau (organisation gouvernementale)
et → TLD : code pays Éthiopie

Q2 — Adresse IP et classe : Commande : nslookup www.anrseb.gov.et → résultat typique :
196.188.120.XX → 1er octet = 196 → **Classe C** (192–223).

Q3 — Routeurs : tracer www.anrseb.gov.et. Serveur localisé en **Éthiopie (Addis-Abeba)**, géré par l'AnRSEB.

Partie 2 — Serveur de messages TCP [15 pts]

■ Architecture : Map<String,String> noms (nom→IP) + Map<String,List<String>> messages. Port suggéré : 5000.

Q1+Q2 — Inscription (étapes 1 et 2)

```
// Client.java – étape 1 (inscription)
import java.net.*;
import java.io.*;

public class Client {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

        String nom = "Alice";
        out.println("inscription " + nom);

        String reponse = in.readLine();
        System.out.println("Réponse serveur : " + reponse);
        socket.close();
    }
}

// Serveur.java – étape 2 (traitement inscription)
import java.net.*;
import java.io.*;
import java.util.*;

public class Serveur {
    static Map<String, String> noms = new HashMap<>();
    static Map<String, List<String>> messages = new HashMap<>();

    public static void main(String[] args) throws Exception {
        ServerSocket serveur = new ServerSocket(5000);
        while (true) {
            Socket client = serveur.accept();
            String ipClient = client.getInetAddress().getHostAddress();
```

```

        BufferedReader in = new BufferedReader(
            new InputStreamReader(client.getInputStream()));
        PrintWriter out = new PrintWriter(
            client.getOutputStream(), true);

        String[] parts = in.readLine().split(" ", 2);
        String commande = parts[0];

        if (commande.equals("inscription")) {
            String nom = parts[1];
            if (noms.containsKey(nom)) {
                out.println("False");
            } else {
                noms.put(nom, ipClient);
                messages.put(nom, new ArrayList<>());
                out.println("True " + noms.keySet());
            }
        }
        // autres commandes ci-dessous ...
        client.close();
    }
}

```

Q4+Q5 — Désinscription (étapes 3 et 4)

```

// Client – envoi désinscription
out.println("désinscription " + nom);
System.out.println("Désinscription : " + in.readLine());

// Serveur – traitement désinscription
} else if (commande.equals("désinscription")) {
    String nom = parts[1];
    if (noms.containsKey(nom) && noms.get(nom).equals(ipClient)) {
        noms.remove(nom);
        messages.remove(nom);
        out.println("True " + noms.keySet());
    } else {
        out.println("False");
    }
}

```

Q7+Q8 — Envoi de message (étapes 5 et 6)

```

// Client – envoi d'un message
out.println("envoi " + msg + " " + destinataire);
System.out.println("Envoi : " + in.readLine()); // True ou False

// Serveur – traitement envoi
} else if (commande.equals("envoi")) {
    int idx = ligne.lastIndexOf(' ');
    String dest = ligne.substring(idx + 1);
    String msg = ligne.substring(6, idx); // après "envoi "
    String exp = getNameByIP(ipClient);
    if (noms.containsKey(dest)) {
        messages.get(dest).add(exp + ": " + msg);
        out.println("True");
    } else {
        out.println("False");
    }
}

```

```

static String getNameByIP(String ip) {
    for (Map.Entry<String,String> e : noms.entrySet())
        if (e.getValue().equals(ip)) return e.getKey();
    return "inconnu";
}

```

Q10+Q11 — Lecture de messages (étapes 7 et 8)

```

// Client – lecture
out.println("lecture");
String ligne;
while ((ligne = in.readLine()) != null && !ligne.equals("FIN"))
    System.out.println("Message reçu : " + ligne);

// Serveur – traitement lecture
} else if (commande.equals("lecture")) {
    String nom = getNameByIP(ipClient);
    if (nom != null && messages.containsKey(nom)) {
        for (String m : messages.get(nom)) out.println(m);
        messages.get(nom).clear();
    }
    out.println("FIN");
}

```

✓ Jeux de test : lancez d'abord le serveur, puis plusieurs clients. Testez les cas nominaux (succès) ET les cas d'erreur (nom déjà existant, destinataire inconnu).

Examen 2024 — Copie de répertoire (UDP)

28 mai 2024 · Protocole UDP · Transfert récursif de répertoires et fichiers

Partie 1 — Réseaux [5 pts]

URL : **www.visa.gov.tj**

www → sous-domaine

visa → nom d'hôte / domaine principal

gov → domaine de second niveau (gouvernemental)

tj → TLD : code pays Tadjikistan

(pas de protocole explicite → HTTP implicite)

Q2 : nslookup www.visa.gov.tj → adresse typique 193.x.x.x → **Classe C**.

Q3 : tracer www.visa.gov.tj → serveur localisé au **Tadjikistan (Douchanbé)**.

Partie 2 — Copie de répertoire UDP [15 pts]

■ Différence clé avec 2023 : UDP (DatagramSocket/DatagramPacket). Pas de connexion établie. Taille max recommandée : 1024 octets par paquet.

Q1+Q2 — Existence du répertoire (étapes 1 et 2)

```
// ClientUDP.java
import java.net.*;

public class ClientUDP {
    static DatagramSocket socket;
    static InetAddress adresse;
    static final int PORT = 9876;

    public static void main(String[] args) throws Exception {
        socket = new DatagramSocket();
        adresse = InetAddress.getByName("localhost");

        envoyer("/home/user/monRepertoire");
        String rep = recevoir();
        System.out.println("Répertoire existe : " + rep);

        if (rep.equals("False")) { socket.close(); return; }
        envoyer("True"); // acquittement → démarre la copie
    }

    static void envoyer(String msg) throws Exception {
        byte[] buf = msg.getBytes();
        socket.send(new DatagramPacket(buf, buf.length, adresse, PORT));
    }

    static String recevoir() throws Exception {
        byte[] buf = new byte[1024];
        DatagramPacket p = new DatagramPacket(buf, buf.length);
        socket.receive(p);
        return new String(p.getData(), 0, p.getLength());
    }
}

// ServeurUDP.java
import java.net.*;
import java.io.*;

public class ServeurUDP {
```

```

static DatagramSocket socket;
static InetAddress clientAddr;
static int clientPort;

public static void main(String[] args) throws Exception {
    socket = new DatagramSocket(9876);
    System.out.println("Serveur UDP démarré sur 9876");

    while (true) {
        String chemin = recevoir();
        File rep = new File(chemin);

        if (rep.exists() && rep.isDirectory()) {
            envoyer("True");
        } else {
            envoyer("False");
            continue;
        }

        if (recevoir().equals("True"))
            explorerRepertoire(rep);
    }
}

static void explorerRepertoire(File rep) throws Exception {
    for (File f : rep.listFiles()) {
        if (f.isDirectory()) {
            envoyer("repertoire " + f.getName());
            recevoir(); // ack
            explorerRepertoire(f);
        } else {
            envoyer("fichier " + f.getName());
            recevoir(); // ack
            envoyerFichier(f);
            recevoir(); // ack fin fichier
        }
    }
    envoyer("fin");
    recevoir(); // ack
}

static void envoyerFichier(File f) throws Exception {
    FileInputStream fis = new FileInputStream(f);
    byte[] bloc = new byte[1024];
    int lu;
    while ((lu = fis.read(bloc)) != -1)
        socket.send(new DatagramPacket(bloc, lu, clientAddr, clientPort));
    fis.close();
    envoyer("EOF");
}

static String recevoir() throws Exception {
    byte[] buf = new byte[1024];
    DatagramPacket p = new DatagramPacket(buf, buf.length);
    socket.receive(p);
    clientAddr = p.getAddress();
    clientPort = p.getPort();
    return new String(p.getData(), 0, p.getLength());
}

```



```

        static void envoyer(String msg) throws Exception {
            byte[] buf = msg.getBytes();
            socket.send(new DatagramPacket(buf, buf.length, clientAddr, clientPort));
        }
    }
}

```

Q7+Q8 — Réception des messages (côté client, étape 5)

```

String msg = recevoir();
if (msg.equals("fin")) {
    System.out.println("Copie terminée.");
    envoyer("True");
} else if (msg.startsWith("repertoire ")) {
    new File("copie/" + msg.substring(11)).mkdirs();
    envoyer("True");
} else if (msg.startsWith("fichier ")) {
    System.out.println("Réception : " + msg.substring(8));
    envoyer("True");
    recevoirFichier("copie/" + msg.substring(8));
    envoyer("True");
}

```

Q11 — Réception d'un fichier par blocs (côté client)

```

static void recevoirFichier(String chemin) throws Exception {
    FileOutputStream fos = new FileOutputStream(chemin);
    while (true) {
        byte[] buf = new byte[1024];
        DatagramPacket p = new DatagramPacket(buf, buf.length);
        socket.receive(p);
        String data = new String(p.getData(), 0, p.getLength());
        if (data.equals("EOF")) break;
        fos.write(p.getData(), 0, p.getLength());
    }
    fos.close();
}

```

✓ Jeux de test : créez une structure répertoire/sous-dossiers/fichiers.txt côté serveur. Vérifiez l'identité de la copie et testez aussi un chemin inexistant.

Examen 2025 — Bibliothèque numérique TCP + Serveur d'heure UDP

19 mai 2025 · TCP (15 pts) + UDP (5 pts) — deux exercices indépendants

Partie 1 — Bibliothèque numérique TCP [15 pts]

■ Structure côté serveur : dossier livres/ contenant des fichiers .txt. Catalogue List<String> avec entrées "Titre;Auteur".
Port suggéré : 6000.

Q1 — Client TCP : connexion et envoi du mot-clé

```
// ClientBiblio.java
import java.net.*;
import java.io.*;

public class ClientBiblio {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 6000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        BufferedReader clavier = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Mot-clé de recherche : ");
        out.println(clavier.readLine());

        String resultats = in.readLine();
        System.out.println("Résultats : " + resultats);
        socket.close();
    }
}
```

Q2 — Serveur TCP : recherche dans le catalogue

```
// ServeurBiblio.java (extrait)
import java.net.*;
import java.io.*;
import java.util.*;

public class ServeurBiblio {
    static List<String> catalogue = Arrays.asList(
        "Les Misérables;Victor Hugo",
        "L'Étranger;Albert Camus",
        "Le Petit Prince;Antoine de Saint-Exupéry",
        "Germinal;Émile Zola"
    );

    public static void main(String[] args) throws Exception {
        ServerSocket serveur = new ServerSocket(6000);
        while (true) {
            Socket client = serveur.accept();
            String ipClient = client.getInetAddress().getHostAddress();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(client.getInputStream()));
            PrintWriter out = new PrintWriter(
                client.getOutputStream(), true);

            // Étapes 2→3 : recherche
            String motCle = in.readLine().toLowerCase();
```

```

List<String> res = new ArrayList<>();
for (String livre : catalogue)
    if (livre.toLowerCase().contains(motCle)) res.add(livre);

out.println(res.isEmpty() ? "Aucun résultat"
    : String.join("|", res));

// Étapes 4→5 : téléchargement
String demande = in.readLine();
File fichier = new File("livres/" + demande + ".txt");

if (fichier.exists()) {
    out.println("OK");
    FileInputStream fis = new FileInputStream(fichier);
    byte[] bloc = new byte[1024];
    int lu;
    OutputStream os = client.getOutputStream();
    while ((lu = fis.read(bloc)) != -1) os.write(bloc, 0, lu);
    fis.close();
    out.println("\n__FIN__");
} else {
    out.println("NOT_FOUND");
}

// Étape 6 : acquittement
String ack = in.readLine();
if ("RECU".equals(ack))
    System.out.println("Livre téléchargé par " + ipClient);

client.close();
}
}
}

```

Q7 — Client : sauvegarde du fichier + acquittement

```

if (in.readLine().equals("OK")) {
    FileOutputStream fos = new FileOutputStream("telechargements/" + titre + ".txt");
    StringBuilder contenu = new StringBuilder();
    InputStream is = socket.getInputStream();
    byte[] buf = new byte[1024];
    int lu;
    while ((lu = is.read(buf)) != -1) {
        String partie = new String(buf, 0, lu);
        if (partie.contains("__FIN__")) {
            contenu.append(partie.replace("\n__FIN__", ""));
            break;
        }
        contenu.append(partie);
    }
    fos.write(contenu.toString().getBytes());
    fos.close();
    out.println("RECU");
} else {
    System.out.println("Livre non trouvé.");
}
}

```

Q10 — Multi-clients (Thread)

```

while (true) {
    Socket client = serveur.accept();

```

```

        new Thread(() -> {
            try { traiterClient(client); }
            catch (Exception e) { e.printStackTrace(); }
        }).start();
    }
}

```

Q12 — Rapport de validation (log console)

```

System.out.println("=== Rapport de validation ===");
System.out.println("Titre téléchargé : " + demande);
System.out.println("Fichier créé      : " + demande + ".txt");
System.out.println("Taille données   : " + fichier.length() + " octets");
System.out.println("Client          : " + ipClient);
System.out.println("Horodatage      : " + new java.util.Date());
System.out.println("=====");

```

Partie 2 — Serveur d'heure UDP [5 pts]

Q1 — ServeurHeureUDP

```

import java.net.*;
import java.time.*;
import java.time.format.*;

public class ServeurHeureUDP {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket(39876);
        System.out.println("Serveur heure UDP démarré sur le port 39876");

        while (true) {
            byte[] buf = new byte[256];
            DatagramPacket requete = new DatagramPacket(buf, buf.length);
            socket.receive(requete);

            String msg = new String(requete.getData(), 0,
                                    requete.getLength().trim());

            byte[] reponse;
            if (msg.equals("HEURE")) {
                String heure = LocalDateTime.now()
                    .format(DateTimeFormatter.ofPattern("HH:mm:ss"));
                reponse = heure.getBytes();
                System.out.println("Heure envoyée : " + heure
                                   + " → " + requete.getAddress());
            } else {
                reponse = "Commande inconnue".getBytes();
            }

            socket.send(new DatagramPacket(reponse, reponse.length,
                                           requete.getAddress(), requete.getPort()));
        }
    }
}

```

Q2 — ClientHeureUDP

```

import java.net.*;

public class ClientHeureUDP {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();
        InetAddress addr = InetAddress.getByName("localhost");
    }
}

```

```

byte[] buf = "HEURE".getBytes();
socket.send(new DatagramPacket(buf, buf.length, addr, 39876));

byte[] rep = new byte[256];
DatagramPacket reponse = new DatagramPacket(rep, rep.length);
socket.receive(reponse);

System.out.println("Heure reçue du serveur : "
    + new String(reponse.getData(), 0, reponse.getLength()));
socket.close();
    }
}

```

Q3 — Logs attendus

```

// Log SERVEUR :
Serveur heure UDP démarré sur le port 39876
Heure envoyée : 14:35:22 → /127.0.0.1
Heure envoyée : 14:35:25 → /127.0.0.1

```

```

// Log CLIENT :
Heure reçue du serveur : 14:35:22

```

✓ Lancez d'abord le serveur, puis exécutez le client plusieurs fois. Les heures doivent correspondre à l'heure système.